

- 평가서식(제출양식)

과 정 명		LangChain을 활용한 AI 에이전트 개발 집중	제 출 자	이원용
교 육 일		2026/06/20 ~ 2026/06/21 (토, 일 주말 2일)		
수 행 결 과	문항	제출 파일 내용		
	1.	다음은 사용자 질문과 답변을 바탕으로 "추천 질문"과 "질문 증강"을 동시에 생성하는 LangChain 프로그램의 일부입니다. 주석과 흐름도를 참고하여 빈칸 (1)부터 (10)에 들어갈 가장 적절한 코드를 작성해 보세요. <pre>from langchain_core.prompts import PromptTemplate from langchain_openai import ChatOpenAI # 병렬 실행을 위한 Runnable 클래스 임포트 from langchain_core.runnables import (1) from langchain_core.output_parsers import (StrOutputParser, CommaSeparatedListOutputParser,) llm = ChatOpenAI(model="gpt-4o-mini") list_parser = CommaSeparatedListOutputParser() # 컴포넌트의 출력 포맷 안내 문구를 가져오는 메서드 format_instruction = list_parser.(2)() # 사용자 질문에 대한 기본 답변 생성 체인 구축 question_text = "인셉션 감독이 누구인가요?" base_template = """ 당신은 영화를 추천해 주는 AI 챗봇입니다. 사용자 질문에 답변하세요. 사용자 질문: {question} """ prompt_base = PromptTemplate(template=base_template) # LCEL 연결 연산자 chain_base = prompt_base (3) llm (3) StrOutputParser() # 체인 실행 메서드 answer_text = chain_base.(4)({"question": question_text}) # 추천 질문 생성 프롬프트 및 체인 정의 template_recommend = """ 당신은 영화를 추천해 주는 AI 챗봇입니다. 사용자의 질문과 AI의 답변을 바탕으로 사용자가 새롭게 질문할 만한 추천 프롬프트 3 개를 생성하시오. 사용자 질문: {question}</pre>		

```
AI 답변: {answer}
```

```
FORMAT: {format}
```

```
"""
```

```
prompt_recommend = PromptTemplate(  
    template=template_recommend,  
    partial_variables={"format": format_instruction}  
)
```

```
# 추천 질문용 출력 파서 변수명
```

```
chain_recommend = prompt_recommend | llm | (5)
```

```
# 질문 증강(Augmentation) 프롬프트 및 체인 정의
```

```
template_augmented = """
```

```
당신은 사용자 질문을 새롭게 생성하는 AI 비서입니다.
```

```
사용자의 질문과 유사하지만 사용된 단어가 다른 새로운 질문 3개를 생성하시오.
```

```
사용자 질문: {question}
```

```
FORMAT: {format}
```

```
"""
```

```
prompt_augmented = PromptTemplate(  
    template=template_augmented,  
    partial_variables={"format": format_instruction}  
)
```

```
chain_augmented = prompt_augmented | llm | list_parser
```

```
# 추천 질문 체인과 질문 증강 체인을 병렬로 묶어서 실행
```

```
agent_parallel_map = (6) (
```

```
    recommend=(7), # 추천 질문 체인 지정
```

```
    augmented=(8) # 질문 증강 체인 지정
```

```
)
```

```
# 병렬 체인 실행 및 결과 받아오기
```

```
execution_output = agent_parallel_map.invoke(  
    "question": question_text,  
    "answer": (9) # 첫 번째 체인의 결과물 변수명  
)
```

```
# 실행 결과 출력
```

```
print("\n추천 질문")
```

```
print("-" * 50)
```

```
# 추천 질문 결과에 접근하기 위한 딕셔너리 키(문자열)
```

```
for x in execution_output[(10)]:
```

```
    print("-", x)
```

	<pre>print("\n증강 질문") print("-" * 50) for x in execution_output["augmented"]: print("-", x)</pre>
답안	1. RunnableParallel 2. get_format_instructions 3. 4. invoke 5. list_parser 6. RunnableParallel 7. chain_recommend 8. chain_augmented 9. answer_text 10. "recommend"

다음은 사용자 질문과 답변을 바탕으로 "추천 질문"과 "질문 증강"을 동시에 생성하는 LangChain 프로그램의 일부입니다. 주석과 흐름도를 참고하여 빈칸 (1)부터 (10)에 들어갈 가장 적절한 코드를 작성해 보세요.

```
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
# 병렬 실행을 위한 Runnable 클래스 импорт
from langchain_core.runnables import (1)
from langchain_core.output_parsers import (
    StrOutputParser,
    CommaSeparatedListOutputParser,
)

llm = ChatOpenAI(model="gpt-4o-mini")
list_parser = CommaSeparatedListOutputParser()
# 컴포넌트의 출력 포맷 안내 문구를 가져오는 메서드
format_instruction = list_parser.(2)()

# 사용자 질문에 대한 기본 답변 생성 체인 구축
question_text = "인셉션 감독이 누구인가요?"
base_template = """
당신은 영화를 추천해 주는 AI 챗봇입니다. 사용자 질문에 답변하세요.
사용자 질문: {question}
"""
prompt_base = PromptTemplate(template=base_template)
# LCEL 연결 연산자
chain_base = prompt_base (3) llm (3) StrOutputParser()
# 체인 실행 메서드
answer_text = chain_base.(4)({"question": question_text})

# 추천 질문 생성 프롬프트 및 체인 정의
template_recommend = """
당신은 영화를 추천해 주는 AI 챗봇입니다.
사용자의 질문과 AI의 답변을 바탕으로 사용자가 새롭게 질문할 만한 추천 프롬프트 3개를 생성하시오.
사용자 질문: {question}
AI 답변: {answer}

FORMAT: {format}
"""
prompt_recommend = PromptTemplate(
    template=template_recommend,
    partial_variables={"format": format_instruction}
)
```

추천 질문용 출력 파서 변수명

```
chain_recommend = prompt_recommend | llm | (5)
```

질문 증강(Augmentation) 프롬프트 및 체인 정의

```
template_augmented = ""
```

당신은 사용자 질문을 새롭게 생성하는 AI 비서입니다.

사용자의 질문과 유사하지만 사용된 단어가 다른 새로운 질문 3개를 생성하십시오.

사용자 질문: {question}

```
FORMAT: {format}
```

```
"""
```

```
prompt_augmented = PromptTemplate(  
    template=template_augmented,  
    partial_variables={"format": format_instruction}  
)
```

```
chain_augmented = prompt_augmented | llm | list_parser
```

추천 질문 체인과 질문 증강 체인을 병렬로 묶어서 실행

```
agent_parallel_map = (6) (  
    recommend=(7), # 추천 질문 체인 지정  
    augmented=(8) # 질문 증강 체인 지정  
)
```

병렬 체인 실행 및 결과 받아오기

```
execution_output = agent_parallel_map.invoke(  
    "question": question_text,  
    "answer": (9) # 첫 번째 체인의 결과물 변수명  
)
```

실행 결과 출력

```
print("\n추천 질문")
```

```
print("-" * 50)
```

추천 질문 결과에 접근하기 위한 딕셔너리 키(문자열)

```
for x in execution_output[(10)]:  
    print("-", x)
```

```
print("\n증강 질문")
```

```
print("-" * 50)
```

```
for x in execution_output["augmented"]:
```

```
    print("-", x)
```